

COMP2271: Computer Graphics

Part 3: Lighting and Shading

Lecturer: Frederick Li
Department of Computer Science
Durham University

How to Learn This Lecture

Lighting is where math meets art. To master this lecture, break the concepts down into components and vectors:

- **Deconstruct the Light:** Do not view lighting as one big equation. Break it down: **Ambient** (Base visibility), **Diffuse** (Shape/Matte), and **Specular** (Shine/Gloss). Visualise how these layers stack on top of each other.
- **Visualise the Dot Product:** The dot product ($\vec{N} \cdot \vec{L}$) is your "brightness dial". If the vectors align, the result is 1 (bright). If they are perpendicular, it is 0 (dark). This single concept drives almost all lighting.
- **The "Where" Defines the Look:** The math (Blinn-Phong) is usually the same, but *where* you calculate it changes the result. Calculating it in the **Vertex Shader** gives you Gouraud (fast, smoothish), while the **Fragment Shader** gives you Phong (expensive, perfect).

1 Section 1: Introduction - The Illusion of Reality

In the previous lectures, we learned how to define 3D objects using meshes and how to transform them into the correct position on our 2D screen. However, a transformed object rendered with a single, flat colour appears artificial and unconvincing. It lacks depth, shape, and substance. To create the illusion of three-dimensionality, we must simulate how light interacts with surfaces in the real world. A 3D model without lighting looks flat; lighting is what reveals its shape, texture, and material properties.



Unlit Sphere (Flat Geometry) Lit Sphere (3D Volume)

Figure 1: The difference between an unlit and a lit object. Without lighting equations, geometry appears flat. Lighting provides crucial visual cues about the object's 3D form.

1.1 Defining Lighting in Computer Graphics

In Computer Graphics (CG), "lighting" does not involve physical photons. Instead, it is a mathematical simulation of electromagnetic radiation and how it transfers energy when intersecting with virtual materials. The goal is to calculate the final colour and luminance of a pixel by evaluating vectors, specifically, the relationships between the light source's position, the camera's viewpoint, and the geometric orientation of the surface (the normal vector).

1.2 Local vs. Global Illumination

When simulating light transport, graphics pipelines must choose between physical accuracy and computational viability. This leads to two primary paradigms:

- **Global Illumination (GI):** This approach simulates the true physics of light, where photons bounce infinitely between surfaces. It accounts for indirect lighting, colour bleeding (e.g., a red wall casting a warm glow on a white floor), and caustics. While it produces stunning, photorealistic results, calculating these infinite bounces is extraordinarily computationally expensive.
- **Local Illumination:** This is a simplified mathematical model. It calculates the light energy reaching a surface *directly* from a light source, completely ignoring light that might bounce off other objects in the scene. If a surface cannot "see" the light source directly, it receives zero light energy and is rendered completely black.

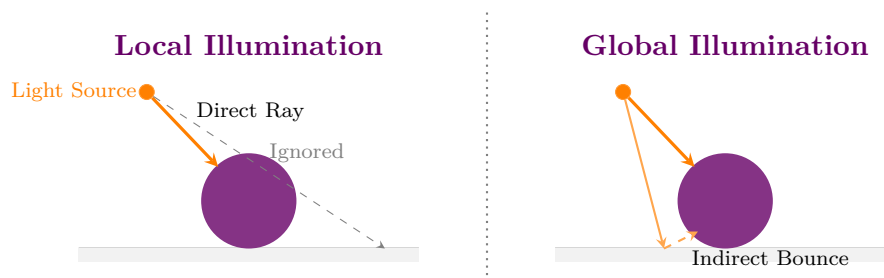


Figure 2: Local Illumination considers only direct light rays, whereas Global Illumination accounts for rays bouncing off surfaces (like the floor) to illuminate shadowed areas.

1.3 Illumination in the Rasterisation Pipeline

Real-time rendering engines (such as those powered by OpenGL, WebGL, or DirectX) traditionally rely on the **rasterisation** pipeline. Rasterisation projects 3D triangles onto a 2D screen one by one.

Because of this "one-triangle-at-a-time" architecture, when the GPU is calculating the colour of a specific pixel (fragment), it only has access to local data: the fragment's normal vector, its material properties, and the vector pointing to the light source. It has *no spatial awareness* of the rest of the scene's geometry at that exact moment.

Therefore, **rasterisation fundamentally relies on Local Illumination**. It cannot natively trace a ray bouncing off a nearby wall because it doesn't "know" where that wall is while shading the current pixel. To compensate for this limitation and prevent shadowed areas from appearing pitch black, real-time graphics rely on "hacks" like ambient lighting.

This lecture explores two fundamental, interconnected concepts to achieve this local illumination:

- **Lighting Model:** A set of mathematical equations used to approximate the complex physics of how light reflects off a surface. This defines *what colour* a point on a surface should be.
- **Shading Model:** The technique for applying a lighting model across a surface. This defines *where and how often* we perform the lighting calculation.

Our journey begins with the classic **Blinn-Phong lighting model**, a simple yet effective approximation that has been a cornerstone of real-time graphics for decades.

2 Section 2: The Blinn-Phong Lighting Model

The Blinn-Phong model is an *empirical model*, meaning it is designed to approximate the visual results of light interaction rather than simulating the underlying physics perfectly. Its success and longevity are due to its computational efficiency and its ability to produce good-looking results. It simplifies the complexities of global illumination by breaking down lighting into three distinct, manageable components, which are then summed together to produce the final colour.

$$\text{Final Colour} = \text{Ambient Component} + \text{Diffuse Component} + \text{Specular Component}$$

We will now examine each of these components in detail.

2.1 Ambient Lighting: The Global Glow

In the real world, light rays bounce off many surfaces before reaching our eyes. This phenomenon, known as *global illumination*, ensures that even areas not in direct line-of-sight of a light source are not completely black. A perfect simulation of this is computationally very expensive.

The ambient component of the Blinn-Phong model is a cheap and effective hack to simulate this indirect lighting.

- **Definition:** Ambient light is a low-intensity, constant light that illuminates every point on every object in the scene equally, regardless of its orientation or position.
- **Purpose:** To ensure that parts of an object in shadow are not rendered as pure black, making them visible.
- **Calculation:** It is the simplest lighting calculation, involving a multiplication of the object's base colour by the ambient light colour, modulated by an ambient strength factor.

$$\text{Ambient Colour} = \text{Ambient Strength} \times \text{Light Colour} \times \text{Object Colour}$$

GLSL Implementation: This is the simplest lighting component to implement in a fragment shader.

```
// Fragment Shader
uniform vec3 u_lightColor;
uniform vec3 u_objectColor;

void main() {
    float ambientStrength = 0.1;
    // A small constant factor
    vec3 ambient = ambientStrength * u_lightColor;

    vec3 result = ambient * u_objectColor;
    gl_FragColor = vec4(result, 1.0);
}
```

Analysis: The ambient component provides a base colour to the object, but on its own, it yields a flat, uninteresting image. It lacks any sense of shape or dimension because the calculation is identical for every fragment of the object. Its true value comes when it is combined with the other components.

2.2 Diffuse Lighting: Revealing Form

Diffuse reflection occurs when light hits a rough or matte surface, causing it to scatter in many directions equally. Think of surfaces like chalk, unpolished wood, or clay. This type of reflection is what primarily reveals the shape of an object.

- **Definition:** The brightness of a diffuse surface is dependent on the angle between the light source's direction and the surface normal.
- **Principle (Lambert's Cosine Law):** A surface is brightest when the light hits it perpendicularly (angle = 0°) and darkest when the light is parallel to it (angle = 90°).
- **Calculation:** This relationship is efficiently calculated using the **dot product** of the normalised light direction vector and the normalised surface normal vector.

The formula for the diffuse component is:

$$\text{Diffuse Colour} = \max(0, \vec{N} \cdot \vec{L}) \times \text{Light Colour} \times \text{Object Colour}$$

Where $\vec{N}$ is the surface normal and $\vec{L}$ is the direction vector from the surface point to the light source. The $\max(0, \dots)$ clamp is crucial to prevent surfaces facing away from the light (where the dot product would be negative) from having "negative" light.

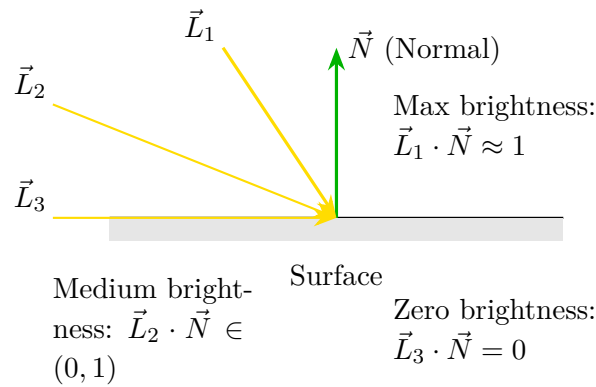


Figure 3: Diffuse lighting intensity is proportional to the cosine of the angle between the light vector ($\vec{L}$) and the surface normal ($\vec{N}$), calculated via the dot product.

GLSL Implementation: The diffuse calculation is performed in the fragment shader. It requires the surface normal, which is typically calculated in the vertex shader and passed as a ‘varying’.

```
// Fragment Shader
uniform vec3 u_lightColor;
uniform vec3 u_objectColor;
uniform vec3 u_lightDirection; // Direction FROM surface TO light
varying vec3 v_normal;
// Normal vector passed from vertex shader

void main() {
    // Normalize vectors to ensure dot product gives cosine of the angle
    vec3 normal = normalize(v_normal);
    vec3 lightDir = normalize(u_lightDirection);

    // Calculate the diffuse intensity factor
    float diff = max(dot(normal, lightDir), 0.0);
    // Compute the final diffuse color
    vec3 diffuse = diff * u_lightColor;

    vec3 result = diffuse * u_objectColor;
    gl_FragColor = vec4(result, 1.0);
}
```

Analysis: Diffuse lighting is the most important component for revealing the 3D shape of an object. As a light source moves around a sphere, the bright spot will move across its surface, clearly defining its curvature. However, it only simulates matte surfaces; it cannot create shiny highlights.

2.3 Specular Lighting: The Shine of a Surface

Specular reflection is the mirror-like reflection of light from a surface, which creates bright highlights. It is characteristic of smooth, shiny materials like polished metal, plastic, or water.

- **Definition:** The specular highlight is brightest when the viewer is perfectly aligned with the reflection of the light source.
- **Principle:** The intensity depends on the angle between the **view vector** and the **reflection vector**. The smaller the angle, the brighter the highlight.
- **Shininess:** A "shininess" or "specular power" exponent is used to control the size and sharpness of the highlight. A higher exponent results in a smaller, more intense highlight, simulating a smoother surface.

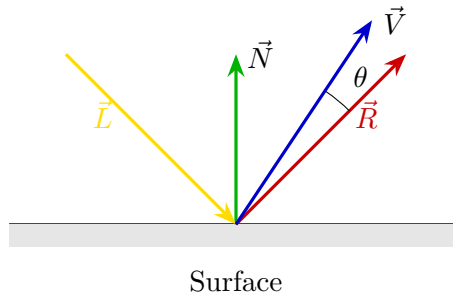


Figure 4: Specular lighting depends on the angle θ between the view vector ($\vec{V}$) and the reflection vector ($\vec{R}$). $\vec{R}$ is calculated by reflecting the light vector ($\vec{L}$) across the normal ($\vec{N}$).

The formula for the Phong specular component is:

$$\text{Specular Colour} = \left(\max(0, \vec{V} \cdot \vec{R}) \right)^\alpha \times \text{Specular Strength} \times \text{Light Colour}$$

Where $\vec{V}$ is the view direction, $\vec{R}$ is the reflection of $\vec{L}$ about $\vec{N}$, and α (alpha) is the shininess exponent.

The Blinn-Phong Optimisation (Halfway Vector) Calculating the reflection vector $\vec{R}$ can be computationally intensive. The Blinn-Phong model introduces a clever optimisation by using a **halfway vector** ($\vec{H}$).

- **Halfway Vector ($\vec{H}$):** This is a unit vector that lies exactly halfway between the light direction vector $\vec{L}$ and the view vector $\vec{V}$.

$$\vec{H} = \frac{\vec{L} + \vec{V}}{\|\vec{L} + \vec{V}\|}$$

- **Principle:** The highlight is at its maximum when this halfway vector aligns with the surface normal $\vec{N}$. This is computationally cheaper than calculating $\vec{R}$ and comparing it to $\vec{V}$, and it often produces more visually pleasing, softer highlights.

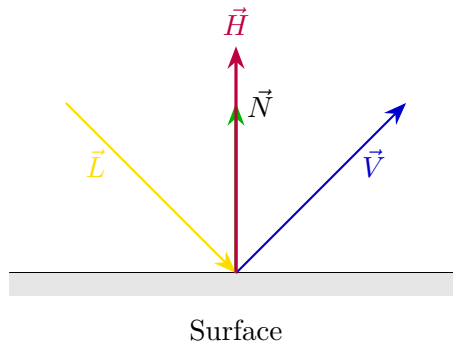


Figure 5: The Blinn-Phong model uses the halfway vector ($\vec{H}$). The highlight is calculated based on the angle between $\vec{H}$ and $\vec{N}$.

The Blinn-Phong specular formula is:

$$\text{Specular Colour} = \left(\max(0, \vec{N} \cdot \vec{H}) \right)^\alpha \times \text{Specular Strength} \times \text{Light Colour}$$

This is the version most commonly implemented in modern real-time graphics when referring to "Blinn-Phong".

GLSL Implementation: This requires more inputs ('uniforms') than the other components, including the camera's position and the fragment's world position to calculate the view vector.

```
// Fragment Shader
uniform vec3 u_viewPosition; // World-space position of the camera
uniform vec3 u_lightPosition;
// World-space position of the light
varying vec3 v_normal; // Interpolated normal from vertex shader
varying vec3 v_fragPosition;
// Interpolated world position from v.s.

void main() {
    // Vectors
    vec3 normal = normalize(v_normal);
    vec3 lightDir = normalize(u_lightPosition - v_fragPosition);
    vec3 viewDir = normalize(u_viewPosition - v_fragPosition);
    vec3 halfwayDir = normalize(lightDir + viewDir);
    // Specular calculation
    float shininess = 32.0;
    float spec = pow(max(dot(normal, halfwayDir), 0.0), shininess);
    // Final specular color
    float specularStrength = 0.5;
    vec3 specular = specularStrength * spec * u_lightColor;
    gl_FragColor = vec4(specular, 1.0);
}
```


Analysis: The specular component adds the final touch of realism, giving a sense of the material's smoothness and quality. By adjusting the 'shininess' and 'specularStrength' values, artists can simulate a wide range of materials from dull plastic to polished metal.

2.4 Combining the Components

The final step is to sum the three individually calculated components. The ambient, diffuse, and specular terms for the light are calculated first, added together, and then this final light value is multiplied by the object's base colour.

Final Blinn-Phong Equation:

$$I_{final} = (k_a I_a) + (k_d (\vec{N} \cdot \vec{L}) I_l) + (k_s (\vec{N} \cdot \vec{H})^\alpha I_l)$$

- I_{final} : The final light intensity.
- k_a, k_d, k_s : Ambient, diffuse, and specular coefficients (strengths).
- I_a, I_l : Ambient and light source intensities (colours).
- $\vec{N}, \vec{L}, \vec{H}$: Normal, Light, and Halfway vectors.
- α : Shininess exponent.

This resulting light colour is then multiplied by the object's albedo (base colour) to get the final fragment colour.

Combined GLSL Implementation:

```
// Fragment Shader (Full Blinn-Phong)
// ... all uniforms and varyings from previous examples ...
uniform vec3 u_objectColor;
void main() {
    // Common vectors
    vec3 normal = normalize(v_normal);
    vec3 lightDir = normalize(u_lightPosition - v_fragPosition);
    vec3 viewDir = normalize(u_viewPosition - v_fragPosition);
    vec3 halfwayDir = normalize(lightDir + viewDir);

    // 1. Ambient
    float ambientStrength = 0.1;
    vec3 ambient = ambientStrength * u_lightColor;

    // 2. Diffuse
    float diff = max(dot(normal, lightDir), 0.0);
    vec3 diffuse = diff * u_lightColor;
```

```

// 3. Specular
float shininess = 32.0;
float specularStrength = 0.5;
float spec = pow(max(dot(normal, halfwayDir), 0.0), shininess);
vec3 specular = specularStrength * spec * u_lightColor;
// Combine and multiply by object color
vec3 result = (ambient + diffuse + specular) * u_objectColor;
gl_FragColor = vec4(result, 1.0);
}

```

2.5 The Importance of Normal Vectors

All of our lighting calculations depend on the **surface normal**. This vector must be accurately provided and correctly transformed.

Defining Normals: Normals are defined per-vertex in our 3D model. For a flat surface like one face of a cube, all vertices for that face will share the same normal vector. For a curved surface like a sphere, each vertex will have a unique normal that points directly outwards from the centre of the sphere.

Transforming Normals: When we apply transformations (like non-uniform scaling) to our model using a model matrix, the normals can become distorted and no longer perpendicular to the surface. Simply multiplying the normal by the model matrix is incorrect. The correct way to transform a normal vector is to multiply it by the **inverse transpose** of the model matrix.

$$\vec{N}_{transformed} = (\mathbf{M}^{-1})^T \vec{N}_{original}$$

This calculation is typically done on the CPU, and the resulting "normal matrix" is passed to the vertex shader as a separate uniform ('mat3' is sufficient as normals are directions).

Vertex Shader Responsibility: The vertex shader's job is to correctly transform the vertex position and the vertex normal, passing them along to the fragment shader.

```

// Vertex Shader
attribute vec4 a_position;
attribute vec3 a_normal;

uniform mat4 u_modelMatrix;
uniform mat4 u_viewMatrix;
uniform mat4 u_projectionMatrix;
uniform mat3 u_normalMatrix;
// The inverse transpose of the model matrix

```

```
varying vec3 v_normal;  
varying vec3 v_fragPosition;  
void main() {  
    // Transform position to clip space  
    gl_Position = u_projectionMatrix * u_viewMatrix * u_modelMatrix * a_position;  
    // Calculate fragment position in world space for the fragment shader  
    v_fragPosition = vec3(u_modelMatrix * a_position);  
    // Transform the normal and pass it to the fragment shader  
    v_normal = u_normalMatrix * a_normal;  
}
```

3 Section 3: Types of Light Sources

In the Blinn-Phong model, we referred to a generic "Light Vector" ($\vec{L}$). However, in a real scene, the nature of this vector changes drastically depending on the type of light source being simulated. There are two primary types of dynamic lights used in real-time graphics: **Directional Lights** and **Point Lights**.

3.1 Directional Light (The Sun)

A directional light represents a light source that is infinitely far away, such as the sun. Because the distance is effectively infinite, light rays arriving at the object are parallel to each other.

- **Geometric Characteristic:** The light has no specific position in the scene, only a **direction**.
- **Mathematical Consequence:** The light vector $\vec{L}$ is **constant**. It is identical for every single vertex and fragment in the scene, regardless of where the object is located.
- **Attenuation:** There is no attenuation (fading) over distance. The sun is just as bright at the top of a mountain as it is at the bottom.

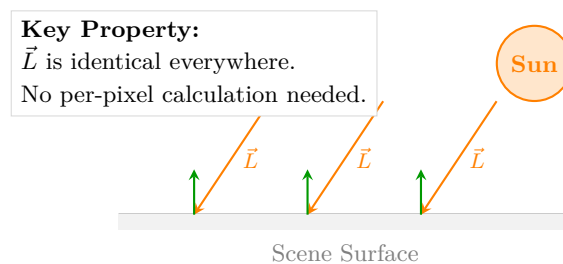


Figure 6: Directional light rays remain parallel. The angle of incidence is constant across a flat surface.

Shader Implementation: Directional lights are the most computationally efficient. Since $\vec{L}$ is constant, we do not need to calculate it inside the shader; we simply pass the fixed direction as a ‘uniform’.

```
// Fragment Shader (Directional)
uniform vec3 u_lightDirection; // Fixed uniform (e.g., -1, -1, -0.5)

void main() {
    // 1. Negate to get vector pointing TO light
    vec3 L = normalize(-u_lightDirection);

    // 2. No distance calculation needed. Proceed to Blinn-Phong...
}
```

3.2 Point Light (The Bulb)

A point light represents a source with a specific 3D position in the world that emits light radially in all directions (e.g., a light bulb, a candle, or a fire).

- **Geometric Characteristic:** The light has a **position**, but no specific direction (it shines everywhere).
- **Mathematical Consequence:** The light vector $\vec{L}$ **changes** for every fragment. We must calculate the vector pointing from the fragment’s position to the light’s position: $\vec{L} = P_{light} - P_{frag}$.
- **Attenuation:** Light intensity diminishes (attenuates) as it travels further from the source.

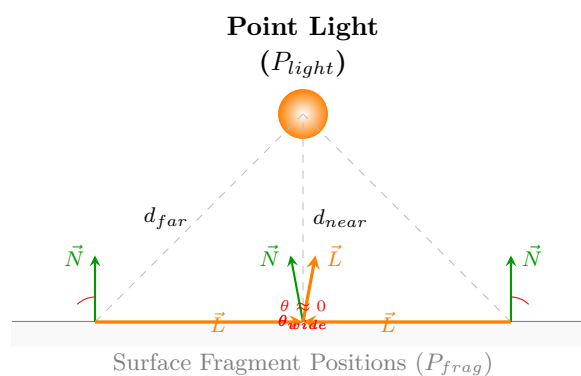


Figure 7: The geometry of Point Lighting. Unlike directional light, the Light Vector ($\vec{L}$) changes direction per-pixel, the incident angle (θ) grows with distance, and the intensity drops over distance (d).

3.2.1 Attenuation: The Inverse Square Law

In physics, light intensity follows the Inverse Square Law: $I \propto \frac{1}{d^2}$. However, in graphics, a pure $1/d^2$ formula is difficult to control artistically (it stays bright very close to the source and then vanishes too abruptly).

To solve this, we use a tunable attenuation formula:

$$F_{att} = \frac{1}{K_c + K_l \cdot d + K_q \cdot d^2}$$

Where d is the distance, and K_c (Constant), K_l (Linear), and K_q (Quadratic) are artist-defined coefficients that control the "falloff" curve.

Shader Implementation (Vertex vs. Fragment): Implementing point lights requires co-operation between stages.

1. **Vertex Shader:** Must calculate the **World Position** of the vertex. Lighting cannot be done in Clip Space or Screen Space; it requires actual 3D distances.
2. **Fragment Shader:** Receives the interpolated world position and calculates the distance and direction for that specific pixel.

```
// --- Vertex Shader ---
varying vec3 v_fragPos; // Output to Fragment

void main() {
    // We transform the vertex to World Space and pass it on
    vec4 worldPos = u_modelMatrix * vec4(a_position, 1.0);
    v_fragPos = vec3(worldPos);

    gl_Position = u_mvpMatrix * vec4(a_position, 1.0);
}

// --- Fragment Shader ---
uniform vec3 u_lightPos; // The position of the light bulb

void main() {
    // 1. Vector Math per-pixel
    vec3 lightVec = u_lightPos - v_fragPos;
    float distance = length(lightVec);
    vec3 L = normalize(lightVec);

    // 2. Calculate Attenuation
    float attenuation = 1.0 / (1.0 + 0.09 * distance + 0.032 * distance * distance);

    // 3. Apply Blinn-Phong, then multiply by attenuation
```

```
    vec3 result = (ambient + diffuse + specular) * attenuation;  
}
```

3.3 Extensions: Spotlights

A Spotlight is essentially a Point Light with a restricted field of view.

- It has a **position** and a **direction**.
- We calculate the angle between the vector pointing to the fragment and the spotlight's forward direction.
- If that angle is outside a defined "cutoff" angle, the light intensity is set to zero.
- To avoid hard edges, a "soft edge" threshold is often added to fade the light out at the boundaries of the cone.

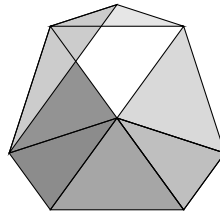
4 Section 4: Shading Models

We have established a *lighting model* (Blinn-Phong) to determine the colour of a point. Now we must decide *how to apply* this model to our triangles. This is the role of the **shading model**. The core question is: *where and how often do we perform the lighting calculation?* The answer has a huge impact on both performance and visual quality. We will explore three main approaches.

4.1 Flat Shading

This is the simplest and fastest shading model.

- **Method:** The lighting calculation is performed only **once for an entire triangle**.
- **Normal Used:** A single "face normal" is computed for the triangle.
- **Result:** Every pixel in the triangle is assigned the exact same colour.
- **Appearance:** This produces a faceted, low-poly look where each individual triangle is clearly visible. It is impossible to create the illusion of a smooth, curved surface.
- **Use Cases:** Useful for stylised, low-polygon art styles or for debugging to visualise the underlying geometry of a mesh.



A sphere rendered with Flat Shading

Figure 8: Flat shading makes no attempt to hide the underlying polygonal mesh.

4.2 Gouraud Shading (Per-Vertex)

Named after Henri Gouraud, this was an early breakthrough that enabled the appearance of smooth surfaces on low-polygon models.

- **Method:** The full Blinn-Phong lighting calculation is performed at **each vertex** of a triangle.
- **Normal Used:** Each vertex has its own smooth normal vector.
- **Process:**
 1. The lighting equation is solved in the **Vertex Shader** to find a colour for each vertex.
 2. These vertex colours are then passed to the rasterizer.
 3. The rasterizer smoothly **interpolates the colours** across the surface of the triangle.
 4. The Fragment Shader receives this interpolated colour and simply outputs it.
- **Appearance:** Produces surfaces that appear smooth. It is much more convincing than flat shading.
- **Limitation:** Specular highlights can be problematic. If a highlight should appear in the middle of a large triangle but not near any of its vertices, it will be missed entirely. This can lead to highlights "popping" in and out as the model moves.

GLSL Implementation (Gouraud):

```
// --- GOURAUD VERTEX SHADER ---
// ... attributes and uniforms ...
varying vec3 v_color;
// Pass a color, not a normal

void main() {
    // ... transform gl_Position and calculate world position ...

    // Perform FULL lighting calculation here for this one vertex
    vec3 normal = normalize(u_normalMatrix * a_normal);
```

```

// ... calculate ambient, diffuse, specular ...
vec3 finalLight = ambient + diffuse + specular;
v_color = finalLight * u_objectColor;
}

// --- GOURAUD FRAGMENT SHADER ---
precision mediump float;
varying vec3 v_color;
// Receive the interpolated color

void main() {
    // The job is simple: just output the interpolated color.
    gl_FragColor = vec4(v_color, 1.0);
}

```

4.3 Phong Shading (Per-Fragment)

Named after Bui Tuong Phong, this is the modern standard for real-time graphics and should not be confused with the Phong *lighting model*. Phong shading produces the highest quality results.

- **Method:** The lighting calculation is performed for **every single fragment (pixel)**.
- **Normal Used:** Instead of interpolating colours, the rasterizer interpolates the **vertex normal vectors** across the surface of the triangle.
- **Process:**
 1. The Vertex Shader transforms the vertex normals and passes them to the rasterizer.
 2. The rasterizer interpolates these normals for each fragment.
 3. The **Fragment Shader** receives an interpolated, high-precision normal for the specific pixel it is shading.
 4. The full Blinn-Phong lighting calculation is performed inside the fragment shader using this interpolated normal.
- **Appearance:** Produces beautifully smooth surfaces and crisp, accurate specular highlights that move realistically across curved objects.
- **Cost:** This is computationally more expensive than Gouraud shading because the complex lighting math is run millions of times per frame (once per pixel) instead of thousands of times (once per vertex). However, modern GPUs are highly optimised for this exact task.

GLSL Implementation (Phong):

```

// --- PHONG VERTEX SHADER ---
// ... attributes and uniforms ...

```



```

varying vec3 v_normal;
// Pass the transformed normal
varying vec3 v_fragPosition; // Pass the world-space position

void main() {
    // ... transform gl_Position ...

    // Just transform and pass the necessary data.
    // No lighting here.
    v_fragPosition = vec3(u_modelMatrix * a_position);
    v_normal = u_normalMatrix * a_normal;
}

// --- PHONG FRAGMENT SHADER ---
precision mediump float;
varying vec3 v_normal;
varying vec3 v_fragPosition;
// ... lighting uniforms ...

void main() {
    // Perform the FULL lighting calculation here, for this one fragment.
    // This is the same combined lighting code we developed earlier.
    vec3 finalLight = calculateBlinnPhongLighting(v_normal, v_fragPosition);
    gl_FragColor = vec4(finalLight * u_objectColor, 1.0);
}

```

4.4 Visual Comparison of Shading Models

The difference between the shading models is most apparent on a low-polygon mesh representing a curved surface.

Table 1: Comparison of Shading Models

Attribute	Flat Shading	Gouraud Shading	Phong Shading
Calculation Point	Per-Triangle	Per-Vertex	Per-Fragment
Quality	Low (Faceted)	Medium (Smooth)	High (Very Smooth)
Performance Cost	Very Low	Low	High
Specular Highlights	Not possible	Poor / Inaccurate	Excellent / Accurate
Modern Standard	No (Artistic choice)	No (Largely obsolete)	Yes

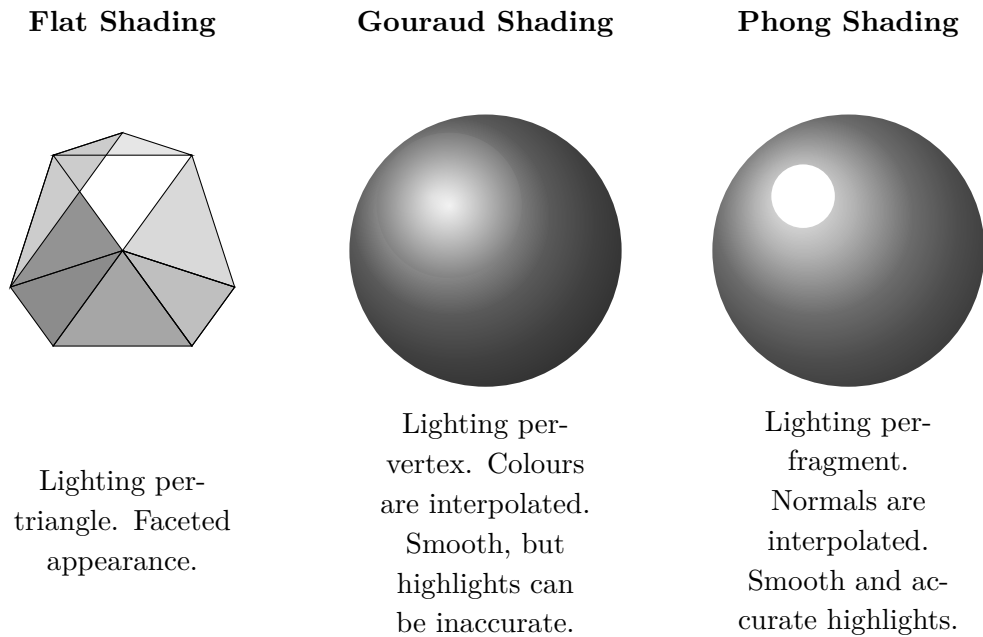


Figure 9: A low-polygon sphere rendered with the three primary shading models.

5 Section 5: Implementation Architecture & Comparisons

While libraries like Three.js abstract the underlying mathematics, a graphics developer must deeply understand the pipeline "division of labour" to optimise performance and visual fidelity. This section consolidates how shading models and lighting components are distributed across the GPU pipeline.

5.1 Shading Models: Vertex vs. Fragment Implementation

The choice of shading model dictates *where* the primary lighting equations are executed.

Feature	Flat Shading	Gouraud Shading	Phong Shading
Calculation Point	Once for an entire triangle.	At each vertex.	Every single fragment (pixel).
Vertex Shader Role	Transforms positions.	Performs full lighting math per vertex.	Transforms and passes normals and positions.
Fragment Shader Role	Outputs one constant colour per face.	Simply outputs the received colour.	Performs full lighting math per pixel.
Interpolation	None (Face constant).	Interpolates colours .	Interpolates normal vectors .
Visual Result	Faceted, clear polygons.	Smooth, but specular highlights can be missed or "pop".	Beautifully smooth with crisp, accurate highlights.

Implementation Watch-Out: Normal Vectors

Transformation: When non-uniform scaling is applied to a model, simply multiplying the normal by the model matrix distorts it. You must multiply the normal by the **inverse transpose** of the model matrix to ensure it remains perpendicular to the surface.

Normalisation: Interpolation alters the length of vectors. When using Phong shading, you must `re-normalize()` the interpolated normal inside the Fragment Shader before using it in dot products.

5.2 Lighting Components: The Blinn-Phong Breakdown

The final colour calculation is a summation of three distinct mathematical components, each requiring specific vectors.

Component	Ambient	Diffuse	Specular (Blinn-Phong)
Physical Meaning	Indirect global illumination.	Matte surface scattering.	Smooth, mirror-like reflection.
Visual Goal	Prevents shadowed areas from rendering pure black.	Reveals the primary 3D shape and curvature.	Simulates material shininess and gloss.
Key Vectors Used	None (Uniform constant).	Surface Normal ($\vec{N}$), Light Direction ($\vec{L}$).	Normal ($\vec{N}$), Halfway Vector ($\vec{H}$).
Core Math	$Strength \times Light \times Object$	$\max(0, \vec{N} \cdot \vec{L})$.	$(\max(0, \vec{N} \cdot \vec{H}))^\alpha$.

Implementation Watch-Out: Vector Math

Clamping: Always use `max(0.0, dot(A, B))` when calculating lighting. If the angle exceeds 90 degrees, the dot product becomes negative, which would erroneously subtract light from your scene.

The Halfway Vector: The Blinn-Phong optimization calculates the halfway vector $\vec{H} = \text{normalize}(\vec{L} + \vec{V})$. The highlight is maximised when $\vec{H}$ aligns perfectly with the surface normal $\vec{N}$.

5.3 High-Level Abstraction: Three.js

While we have analysed the raw GLSL requirements, modern web development relies heavily on frameworks. A library like Three.js provides high-level abstractions that manage these complex GLSL pipelines for you.

The Three.js Approach: Instead of writing vertex and fragment shaders manually, you assign a **Material** to a mesh. The following JavaScript code creates a lit sphere without requiring a single line of raw GLSL:

```
// Assume 'scene' is a THREE.Scene object

// 1. The Material (Acts as the Shader Factory)
// MeshPhongMaterial internally generates the per-fragment Phong shading GLSL
const material = new THREE.MeshPhongMaterial({
  color: 0xddddddd,    // Maps to u_objectColor (Diffuse base)
  specular: 0x111111, // Maps to specular strength
  shininess: 30        // Maps to the exponent alpha
});
const sphereGeometry = new THREE.SphereGeometry(1, 32, 32);
const sphere = new THREE.Mesh(sphereGeometry, material);
scene.add(sphere);

// 2. The Lights (Automatically passed as Uniforms)
const light = new THREE.PointLight(0xffffff, 1, 100);
light.position.set(10, 10, 10);
scene.add(light); // Provides the Light Position for L vector math

const ambientLight = new THREE.AmbientLight(0x404040);
scene.add(ambientLight); // Provides the Ambient color base
```

Analysis: This demonstrates the power of architectural abstraction. The `MeshPhongMaterial` internally contains the complete Phong shading model and GLSL code. The `PointLight` and `AmbientLight` objects automatically pass their positions and colours to the GPU as **uniforms**. This allows the developer to focus on scene composition while the library safely handles the complex vector mathematics and normal matrix transformations.

5.4 Manual Implementation: Writing Raw GLSL

While high-level libraries manage the heavy lifting, custom visual effects or specialized rendering techniques require writing the raw GLSL code directly. When manually implementing shaders, the developer takes full responsibility for the data pipeline between the CPU and GPU, as well as the spatial vector mathematics.

The Data Flow (CPU to GPU): To manually calculate lighting, you must explicitly manage three types of variables:

- **Attributes:** Per-vertex data defined in your 3D model, such as positions and vertex normals, fed directly into the Vertex Shader.
- **Uniforms:** Global, constant data for the current draw call (e.g., light positions, camera positions, transformation matrices) sent from the CPU to either shader.
- **Varyings:** Variables output by the Vertex Shader that are smoothly interpolated by the rasterizer before being handed to the Fragment Shader (e.g., interpolated normals, fragment

positions).

```
// Typical manual pipeline bridge
// --- Vertex Shader ---
attribute vec3 a_normal;
uniform mat3 u_normalMatrix; //
varying vec3 v_normal;      // Interpolated normal

void main() {
    v_normal = u_normalMatrix * a_normal; //
}

// --- Fragment Shader ---
varying vec3 v_normal; //
// Must be re-normalized before use!
```

Developer Watch-Out: Manual Shader Pitfalls

When bypassing abstractions, developers must be vigilant about several mathematical and architectural pitfalls:

- **Coordinate Space Consistency:** Lighting calculations require actual 3D distances and cannot be done in Clip Space or Screen Space. You must explicitly calculate the **World Position** of the fragment by multiplying the vertex position by the model matrix before performing light vector math.
- **The Normal Matrix Distortion:** If your model undergoes non-uniform scaling, simply multiplying normals by the standard Model Matrix will distort them so they are no longer perpendicular to the surface. You must compute the **inverse transpose** of the model matrix on the CPU and pass it as a separate `mat3` uniform.
- **Re-normalisation:** The rasterizer's interpolation process alters the length of vectors. An interpolated normal vector received by the Fragment Shader must be explicitly re-normalized (e.g., `vec3 normal = normalize(v_normal);`) before calculating dot products to ensure Lambert's Cosine Law works correctly.
- **Performance Overhead:** Running complex lighting models per-pixel (Phong Shading) runs millions of times per frame and is highly expensive compared to per-vertex calculations. Ensure that operations that only need to happen once per vertex (like transforming the normal) are kept in the Vertex Shader rather than the Fragment Shader.

6 Section 6: The Architecture of Lighting in WebGL

While high-level libraries like `three.js` allow us to simply add a light source and a material, understanding what happens "under the hood" is vital for debugging and performance optimisation. The rendering process is a coordinated effort between the CPU (Central Processing Unit) and the GPU (Graphics Processing Unit).

6.1 The Pipeline: From Logic to Light

We can visualize the lighting pipeline as a factory line. The CPU acts as the architect, preparing the blueprints and materials, while the GPU acts as the construction crew, building the image pixel-by-pixel in parallel.

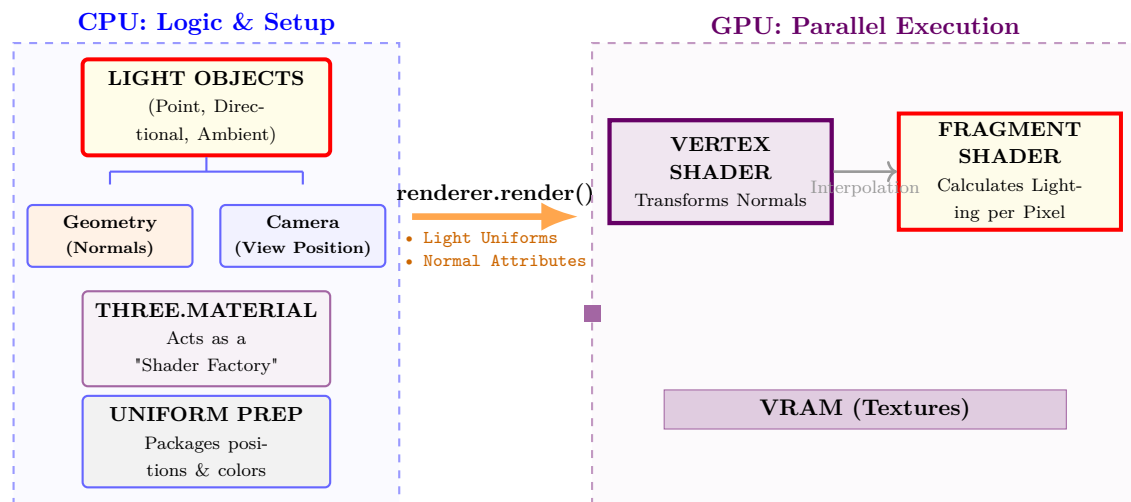


Figure 10: The Lighting Pipeline. The CPU prepares the data (Light positions, Material properties), while the GPU's Fragment Shader performs the heavy lifting of calculating the lighting equation for every pixel.

6.2 Roles and Responsibilities

1. **The Architect (CPU - Three.js):** When you define a 'PointLight' or a 'MeshPhongMaterial', the CPU does not calculate lighting. Instead, it generates the appropriate GLSL code (the "Shader") and packages the data.
 - **Uniforms:** Global data that doesn't change between vertices (e.g., Light Position, Light Colour, Camera Position).
 - **Attributes:** Per-vertex data (e.g., Vertex Normals).
2. **The Courier (Transfer):** When 'renderer.render()' is called, this data is streamed to the GPU. This is why we avoid creating new materials or geometries inside the render loop; it clogs the bridge.
3. **The Worker (GPU - Fragment Shader):** The actual Blinn-Phong math happens here. The Fragment Shader receives the *interpolated* normal vector for the specific pixel it is currently drawing and combines it with the light uniforms to determine the final colour.

7 Section 7: Beyond Realism: Custom Shaders

Standard lighting models like Blinn-Phong strive for *Photorealism*, trying to mimic how physics works. However, computer graphics is also an art form. By modifying the lighting equations,

we can create **Non-Photorealistic Rendering (NPR)** styles, such as cartoons or technical illustrations.

7.1 Cartoon (Toon) Shading

To achieve a "hand-drawn" or "cel-shaded" look, we deliberately discard the smooth gradients of physical light. Instead of allowing the light intensity to fade smoothly from bright to dark, we **quantise** it into discrete bands.

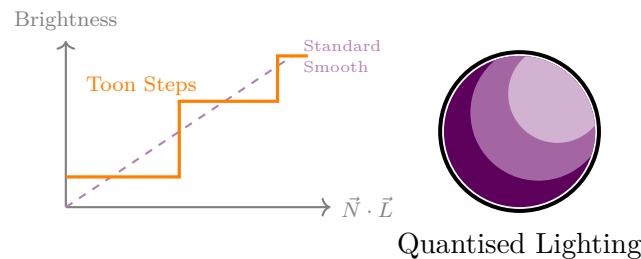


Figure 11: Toon Shading replaces the smooth cosine gradient with a step function.

```
// GLSL: Toon Shading Logic
float intensity = max(0.0, dot(normal, lightDir));

// Quantise the continuous intensity into 3 discrete bands
if (intensity > 0.95)    intensity = 1.0; // Sharp Highlight
else if (intensity > 0.5) intensity = 0.7; // Mid-tone
else                    intensity = 0.2; // Shadow Base

gl_FragColor = vec4(u_color * intensity, 1.0);
```

7.2 Gooch Shading

In standard lighting, shadows fade to black, which hides geometric detail. This is undesirable in engineering diagrams. **Gooch Shading** solves this by mapping light intensity to **Colour Temperature** (Warm vs. Cool) rather than brightness (White vs. Black).

- **Surfaces facing the light:** Rendered in "Warm" colours (e.g., Yellow/Orange).
- **Surfaces away from light:** Rendered in "Cool" colours (e.g., Blue/Purple).
- **Result:** The object is fully visible even in the "shadows," providing better shape clarity.

```
// GLSL: Gooch Shading Logic
float NdotL = dot(normal, lightDir);

// Remap [-1, 1] range to [0, 1] for interpolation
```

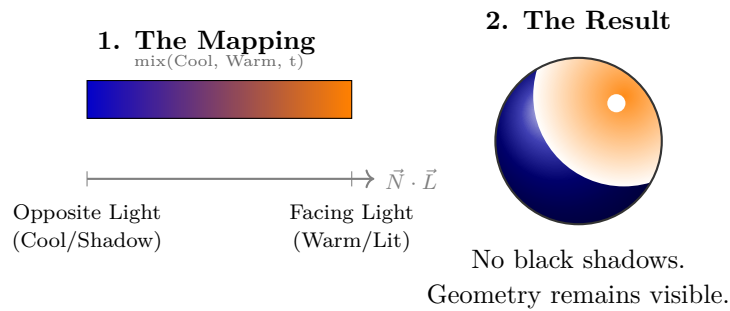


Figure 12: Gooch Shading visualisation. Instead of fading to black, the shadow side fades to a cool blue, ensuring geometric details are never lost in darkness.

```
// 0.0 = Shadow side, 1.0 = Lit side
float t = (NdotL + 1.0) * 0.5;

vec3 cool = vec3(0.0, 0.0, 0.5); // Deep Blue (Shadow)
vec3 warm = vec3(1.0, 0.6, 0.0); // Orange (Lit)

// Interpolate based on angle
vec3 finalColor = mix(cool, warm, t);
```

7.3 Theoretical Framework - Designing NPR Shaders

While photorealistic models like Blinn-Phong strive to mimic the physics of light interaction, Non-Photorealistic Rendering (NPR) prioritises **expressive communication** and **stylistic intent**. Regardless of the specific aesthetic, most NPR shaders operate by deconstructing the standard lighting pipeline into three distinct logical pillars.

The NPR Logic Loop

In professional NPR pipelines, we do not view lighting as a final value, but as a **data stream** that we can filter, warp, and remap to achieve specific artistic goals.

7.3.1 The Control Signal (The Input)

The foundation of most NPR effects remains the dot product of the surface normal and the light direction ($\vec{N} \cdot \vec{L}$).

- **Photorealistic Use:** This value represents the physical attenuation of light according to Lambert's Cosine Law.
- **NPR Use:** This is treated as a **scalar control signal**. It acts as a coordinate that maps every point on the surface to a position on a 1D gradient defining exactly where a fragment sits between "maximum illumination" and "complete occlusion".

7.3.2 The Transfer Function (The Mapping)

This stage defines the "character" of the shader by applying a mathematical filter to the linear control signal.

- **Quantisation (Toon/Cel Shading):** By applying step functions, we discard the smooth cosine gradient. The shader "snaps" continuous values into discrete bands (e.g., if intensity > 0.5 , use 0.7; else use 0.2). This mimics the limited ink/paint palettes used in traditional animation.
- **Biased Interpolation (Gooch/Technical):** We remap the standard range of the dot product. While $\vec{N} \cdot \vec{L}$ usually clamps negative values to zero, technical shaders often remap the $[-1, 1]$ range to $[0, 1]$ using:

$$t = \frac{(\vec{N} \cdot \vec{L}) + 1.0}{2.0}$$

This ensures that even surfaces facing 180° away from the light have a unique value ($t = 0$) rather than being crushed to black.

7.3.3 The Output Mapping (The Final Look)

Finally, the shader maps the filtered signal to a visual property other than simple brightness.

- **Value Mapping:** The signal scales the **luminance** of the base albedo. This is seen in Toon shading where "shadow" is simply a darker version of the object's base colour.
- **Hue/Temperature Mapping:** The signal dictates **chromaticity**. In Gooch shading, the t value is used to linearly interpolate (**mix**) between a "Cool" colour (e.g., Blue) and a "Warm" colour (e.g., Orange). This ensures geometric details remain visible in the shadows because the "dark" side is a different hue, not just a lower intensity.

Component	Photorealism	Toon Shading	Gooch Shading
Logic Goal	Mimic Physical Reality	Stylised Simplification	Geometric Shape Clarity
Gradient	Smooth Linear Cosine	Quantized Step Function	Linear Colour Ramp
Shadows	Fades toward Black	Discrete Dark Blocks	Fades toward Cool Blue
Key Function	$I = \max(0, \vec{N} \cdot \vec{L})$	$I = \text{quantize}(\vec{N} \cdot \vec{L})$	$C = \text{mix}(\text{Cool}, \text{Warm}, t)$

8 Reflective Questions: Lighting the Way Forward

As we conclude our exploration of the Blinn-Phong model and shading techniques, consider the following questions. They are designed to challenge your understanding of the current models and prepare you for the complexities of modern Physically Based Rendering (PBR).

- **The "Phantom" Light (Shadows vs. Lighting):** We calculated diffuse lighting based solely on the angle between the light direction and the surface normal ($\vec{N} \cdot \vec{L}$). *If a large wall stands between the light source and our object, the math still returns a positive number, lighting up the object even though it should be in darkness. Why doesn't our current formula account for blocked light, and what kind of check would we need to perform to create true cast shadows?*
- **The "Plastic" Problem (Energy Conservation):** Blinn-Phong is famous for making objects look slightly like plastic. In the real world, materials like metal or skin reflect light differently depending on the viewing angle (the Fresnel effect). *Since Blinn-Phong is just an approximate math trick, it doesn't guarantee that the light leaving the surface is less than or equal to the light hitting it. How might this lack of "Energy Conservation" break the immersion in a photorealistic scene?*
- **The 4K Challenge (Optimisation):** We established that Phong Shading yields the best results but runs on every single pixel. *On a 4K monitor (3840×2160), there are over 8 million pixels. If we render a simple cube (12 triangles), Gouraud shading runs 8 times, but Phong shading runs 8,000,000 times. Is the visual improvement always worth this massive cost? How might mobile games handle this trade-off?*
- **The Ambient "Hack" (Global Illumination):** We used a single constant colour for Ambient light to prevent shadows from being pitch black. *In reality, shadows are illuminated by light bouncing off nearby surfaces (e.g., a red carpet reflecting red light onto a white wall). How could we simulate this "colour bleeding" without the expensive process of tracing billions of individual light rays?*

"Light thinks it travels faster than anything but it is wrong. No matter how fast light travels, it finds the darkness has always got there first, and is waiting for it."

Terry Pratchett, *Reaper Man*